

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Динамические списки»

Выполнили:
студенты групп 24ВВВ3

Савин А. В.
Назаров Н. Д.
Майер В. С.

Приняли:
Юрова О. В.
Деев М. В.

Пенза 2025

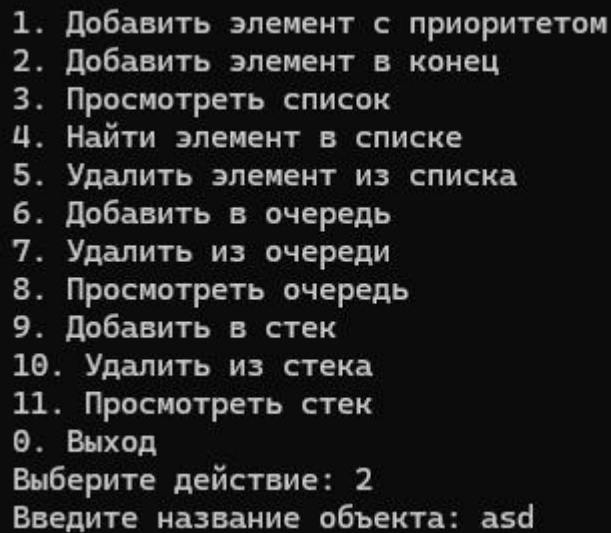
Цель работы

Освоить на практике работу с динамическими списками. Закрепить навыки, написанием разных структур данных.

Лабораторное задание

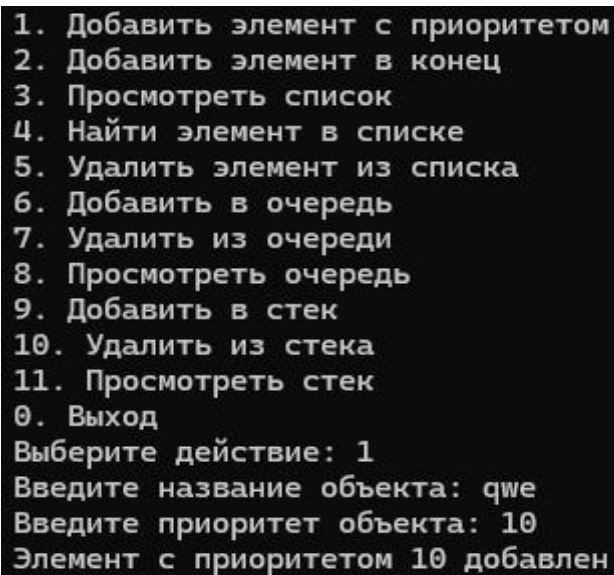
1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. На основе приведенного кода реализуйте структуру данных Очередь.
3. На основе приведенного кода реализуйте структуру данных Стек.

Работа программы



```
1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 2
Введите название объекта: asd
```

Рисунок 1 - Добавление элемента с обычным приоритетом



```
1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 1
Введите название объекта: qwe
Введите приоритет объекта: 10
Элемент с приоритетом 10 добавлен
```

Рисунок 2 - Добавление элемента с приоритетом

```
1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 3
Список:
Имя: qwe, Приоритет: 10
Имя: asd, Приоритет: 0
```

Рисунок 3 - Вывод списка

```
1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 6
Введите название объекта: qwe

1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 6
Введите название объекта: rty
```

Рисунок 4 - Добавление в очередь

```
1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 7
Удален из очереди: qwe
```

Рисунок 5 - Удаление элемента из очереди

```
1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 9
Введите название объекта: sss

1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 9
Введите название объекта: erre
```

Рисунок 6 - Добавление в стек

```
1. Добавить элемент с приоритетом
2. Добавить элемент в конец
3. Просмотреть список
4. Найти элемент в списке
5. Удалить элемент из списка
6. Добавить в очередь
7. Удалить из очереди
8. Просмотреть очередь
9. Добавить в стек
10. Удалить из стека
11. Просмотреть стек
0. Выход
Выберите действие: 10
Удален из стека: erre
```

Рисунок 7 - удаление из стека

Вывод

Освоили на практике работу с динамическими списками. Закрепить навыки, написанием разных структур данных. Написали рабочий код, реализующий структуры данных стек, очередь.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <conio.h>

struct node {
    char inf[256];
    int priority;
    struct node* next;
};

struct node* head = NULL, * last = NULL;
struct node* queue_head = NULL, * queue_last = NULL;
struct node* stack_head = NULL;
```

```

int dlinna = 0;
int queue_length = 0;
int stack_length = 0;

void queue_enqueue(void);
void queue_dequeue(void);
void stack_push(void);
void stack_pop(void);
void review_queue(void);
void review_stack(void);

void spstore_priority(void);
void spstore(void);
void review(void);
void del(char* name);
struct node* find(char* name);
struct node* get_struct(void);

struct node* get_struct_priority(void) {
    struct node* p = NULL;
    char s[256];
    int pr;

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL) {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%s", s);

    printf("Введите приоритет объекта: ");
    scanf("%d", &pr);

    if (*s == 0) {
        printf("Запись не была произведена\n");
    }
}

```

```

        free(p);

        return NULL;
    }

    strcpy(p->inf, s);
    p->priority = pr;
    p->next = NULL;

    return p;
}

struct node* get_struct(void) {
    struct node* p = NULL;
    char s[256];

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL) {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%s", s);

    if (*s == 0) {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }

    strcpy(p->inf, s);
    p->priority = 0;
    p->next = NULL;

    return p;
}

```

```

struct node* get_struct_simple(void) {
    struct node* p = NULL;
    char s[256];

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL) {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%s", s);

    if (*s == 0) {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }

    strcpy(p->inf, s);
    p->priority = 0;
    p->next = NULL;

    return p;
}

void queue_enqueue(void) {
    struct node* p = get_struct_simple();
    if (p == NULL) return;

    if (queue_head == NULL) {
        queue_head = p;
        queue_last = p;
    } else {
        queue_last->next = p;
        queue_last = p;
    }
}

```



```

        queue_length++;
    }

void queue_dequeue(void) {
    if (queue_head == NULL) {
        printf("Очередь пуста\n");
        return;
    }

    struct node* temp = queue_head;
    printf("Удален из очереди: %s\n", temp->inf);

    queue_head = queue_head->next;
    free(temp);

    if (queue_head == NULL) {
        queue_last = NULL;
    }

    queue_length--;
}

void stack_push(void) {
    struct node* p = get_struct_simple();
    if (p == NULL) return;

    p->next = stack_head;
    stack_head = p;

    stack_length++;
}

void stack_pop(void) {
    if (stack_head == NULL) {
        printf("Стек пуст\n");
    }
}

```

```

        return;
    }

    struct node* temp = stack_head;
    printf("Удален из стека: %s\n", temp->inf);

    stack_head = stack_head->next;
    free(temp);

    stack_length--;
}

void review_queue(void) {
    struct node* struc = queue_head;
    if (queue_head == NULL) {
        printf("Очередь пуста\n");
        return;
    }

    printf("Очередь:\n");
    while (struc) {
        printf("Имя: %s\n", struc->inf);
        struc = struc->next;
    }
}

void review_stack(void) {
    struct node* struc = stack_head;
    if (stack_head == NULL) {
        printf("Стек пуст\n");
        return;
    }

    printf("Стек:\n");
    while (struc) {
        printf("Имя: %s\n", struc->inf);
    }
}

```

```

        struc = struc->next;
    }
}

```

```

void spstore_priority(void) {
    struct node* p = NULL;
    struct node* current = NULL;
    struct node* prev = NULL;

    p = get_struct_priority();
    if (p == NULL) {
        return;
    }

    if (head == NULL || p->priority > head->priority) {
        p->next = head;
        head = p;
        if (head->next == NULL) {
            last = head;
        }
    }
    else {
        current = head;
        while (current != NULL && current->priority >= p->priority) {
            prev = current;
            current = current->next;
        }

        prev->next = p;
        p->next = current;

        if (current == NULL) {
            last = p;
        }
    }
}

```

```

        dlinna++;
        printf("Элемент с приоритетом %d добавлен\n", p->priority);
    }

void spstore(void) {
    struct node* p = NULL;
    p = get_struct();
    if (head == NULL && p != NULL) {
        head = p;
        last = p;
    }
    else if (head != NULL && p != NULL) {
        last->next = p;
        last = p;
    }
    return;
}

void review(void) {
    struct node* struc = head;
    if (head == NULL) {
        printf("Основной список пуст\n");
        return;
    }

    printf("Список:\n");
    while (struc) {
        printf("Имя: %s, Приоритет: %d\n", struc->inf, struc->priority);
        struc = struc->next;
    }
}

struct node* find(char* name) {
    struct node* struc = head;
    if (head == NULL) {
        printf("Список пуст\n");
    }
}

```

```

    }

    while (struc) {
        if (strcmp(name, struc->inf) == 0) {
            return struc;
        }
        struc = struc->next;
    }

    printf("Элемент не найден\n");
    return NULL;
}

```

```

void del(char* name) {
    struct node* struc = head;
    struct node* prev = NULL;
    int flag = 0;

    if (head == NULL) {
        printf("Список пуст\n");
        return;
    }

    if (strcmp(name, struc->inf) == 0) {
        flag = 1;
        head = struc->next;
        free(struc);
        struc = head;
    }
    else {
        prev = struc;
        struc = struc->next;
    }

    while (struc) {
        if (strcmp(name, struc->inf) == 0) {
            flag = 1;
            if (struc->next) {

```

```

        prev->next = struc->next;
        free(struc);
        struc = prev->next;
    }
    else {
        prev->next = NULL;
        free(struc);
        return;
    }
}

else {
    prev = struc;
    struc = struc->next;
}
}

if (flag == 0) {
    printf("Элемент не найден\n");
    return;
}

}

int main(void) {
    setlocale(LC_ALL, "Russian");
    int choice;
    char name[256];

    do {
        printf("\n");
        printf("1. Добавить элемент с приоритетом\n");
        printf("2. Добавить элемент в конец\n");
        printf("3. Просмотреть список\n");
        printf("4. Найти элемент в списке\n");
        printf("5. Удалить элемент из списка\n");
        printf("6. Добавить в очередь\n");
        printf("7. Удалить из очереди\n");
    }
}

```

```
printf("8. Просмотреть очередь\n");
printf("9. Добавить в стек\n");
printf("10. Удалить из стека\n");
printf("11. Просмотреть стек\n");
printf("0. Выход\n");
printf("Выберите действие: ");
scanf("%d", &choice);

switch (choice) {
case 1:
    spstore_priority();
    break;
case 2:
    spstore();
    break;
case 3:
    review();
    break;
case 4:
    printf("Введите имя для поиска: ");
    scanf("%s", name);
    find(name);
    break;
case 5:
    printf("Введите имя для удаления: ");
    scanf("%s", name);
    del(name);
    break;
case 6:
    queue_enqueue();
    break;
case 7:
    queue_dequeue();
    break;
case 8:
    review_queue();
```

```
        break;
    case 9:
        stack_push();
        break;
    case 10:
        stack_pop();
        break;
    case 11:
        review_stack();
        break;
    case 0:
        break;
    default:
        break;
}
} while (choice != 0);

_getch();
return 0;
}
```